DOCUMENTO WINDOWS SERVER

Nacho Abad Rodríguez

Índice

1. Introducción 2
2. Objetivos 3
3. Contenidos 4
3.1 Creación general del dominio 4
3.1.1 Windows Server 4
3.1.2 Máquina Cliente 9
3.1.3 Creación de usuarios 11
3.1.4 Comprobaciones 12
3.2 Automatización de Gestión de Usuarios 13
3.2.1 Instalar MySQL/MariaDB en Windows Server 14
3.2.2 Crear la base de datos y tablas 15
3.2.3 Instalar el conector MySQL para PowerShell y crear Scripts 17
3.2.4 Formulario Web y conectividad con la base de datos 21
3.2.5 Programador de Tareas 28
3.2.6 Resolución de Errores 31
4. Conclusión 33
5. Bibliografía 34

1. Introducción

Este proyecto tiene como objetivo montar un entorno completo de dominio basado en Windows Server junto con un cliente Windows 10, simulando la infraestructura básica de una empresa y así probar las habilidades de uno mismo para desarrollar la creatividad de ideas y recrearlas en la realidad. A lo largo del documento se detalla el proceso de configuración del controlador de dominio, la integración de equipos cliente y la gestión inicial de usuarios mediante Active Directory.

Además, se desarrolla una capa de automatización orientada a la gestión del ciclo de vida de los usuarios (altas y bajas), combinando varias tecnologías como MariaDB, PowerShell y un formulario web en PHP. La idea principal es construir un flujo funcional donde las solicitudes se registran en una base de datos y se procesan automáticamente mediante scripts, reduciendo la intervención manual.

Todo el entorno está diseñado con fines de aprendizaje y pruebas, utilizando máquinas virtuales, por lo que no sigue necesariamente criterios de producción, pero sí busca replicar escenarios reales de administración de sistemas. Las contraseñas y usuarios son perfectamente utilizables, pero no recomendables en un entorno real ya que son muy básicas y fáciles de recordar.

2. Objetivos

- Crear un dominio en Windows Server.
- Añadir un equipo Windows 10 al dominio.
- Crear usuarios y organizar UO y grupos.
- Instalar MariaDB/MySQL para guardar solicitudes.
- Crear scripts en PowerShell para altas y bajas.
- Hacer un formulario web para enviar solicitudes.
- Automatizar todo con el Programador de Tareas.
- Comprobar que el sistema funciona correctamente.

3. Contenidos

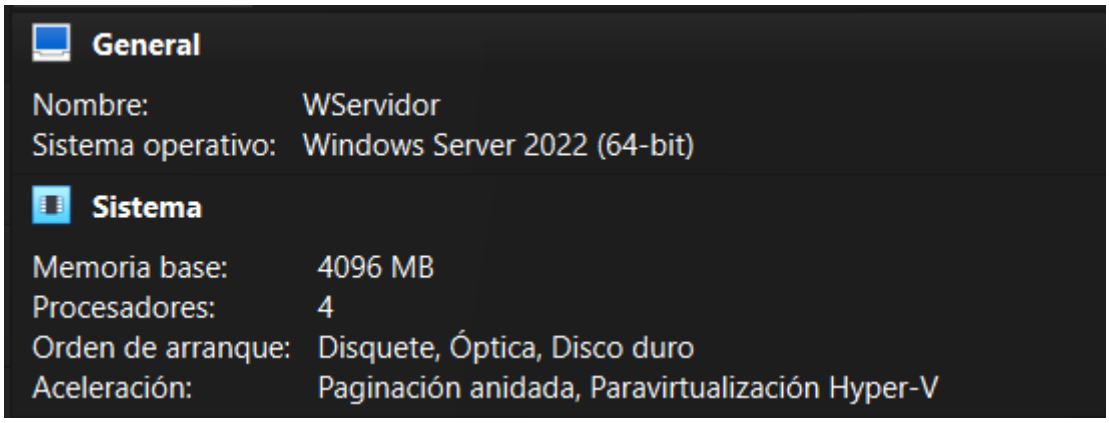
3.1 Creación general del dominio

Todo este proceso se va a realizar con máquinas virtuales, en nuestro caso vamos a utilizar Oracle VirtualBox. Vamos a utilizar dos:

- Windows Server
- W10 Cliente

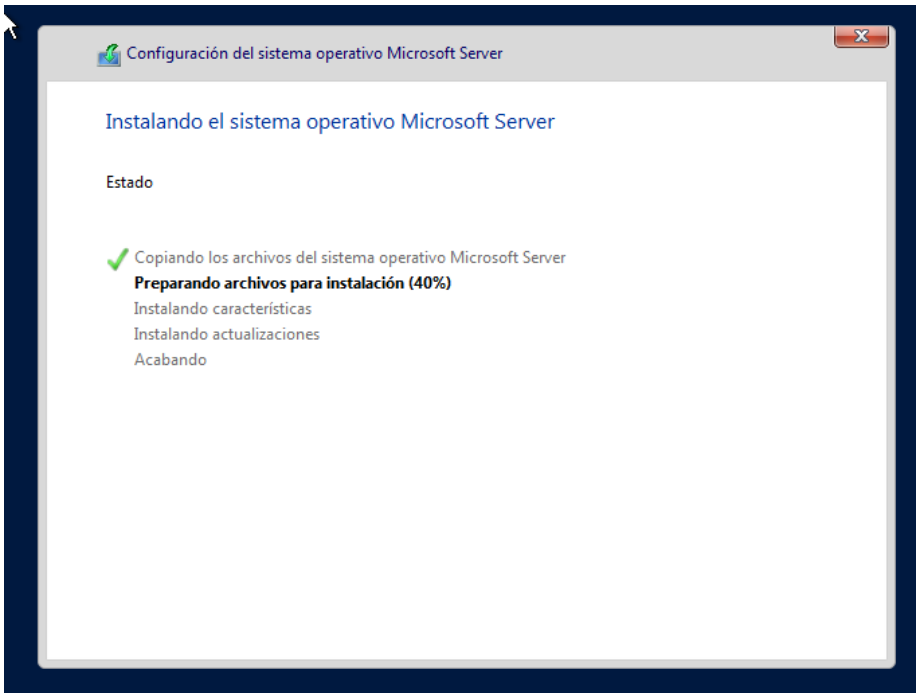
3.1.1 Windows Server

Vamos a crear una máquina con la iso de Windows server 2022. Estas son las especificaciones de sistema que tenemos:



Con un disco de 50Gb nos servirá por ahora. Pondremos un adaptador en red interna, donde pondremos el resto de las máquinas también, para no interferir con nuestra máquina anfitrión.

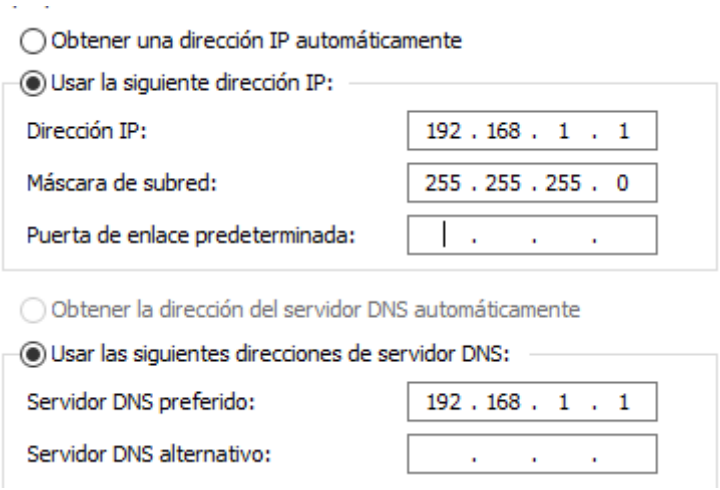
Iniciamos la máquina y procedemos con una instalación normal



Vamos a utilizar la contraseña “Server22.” para el usuario “Administrador”.

Ya tenemos el panel de control principal, pero vamos a tener que activar y configurar un par de opciones antes de promover esta máquina a controlador de dominio y crear este mismo.

Primero configuramos una IP fija:

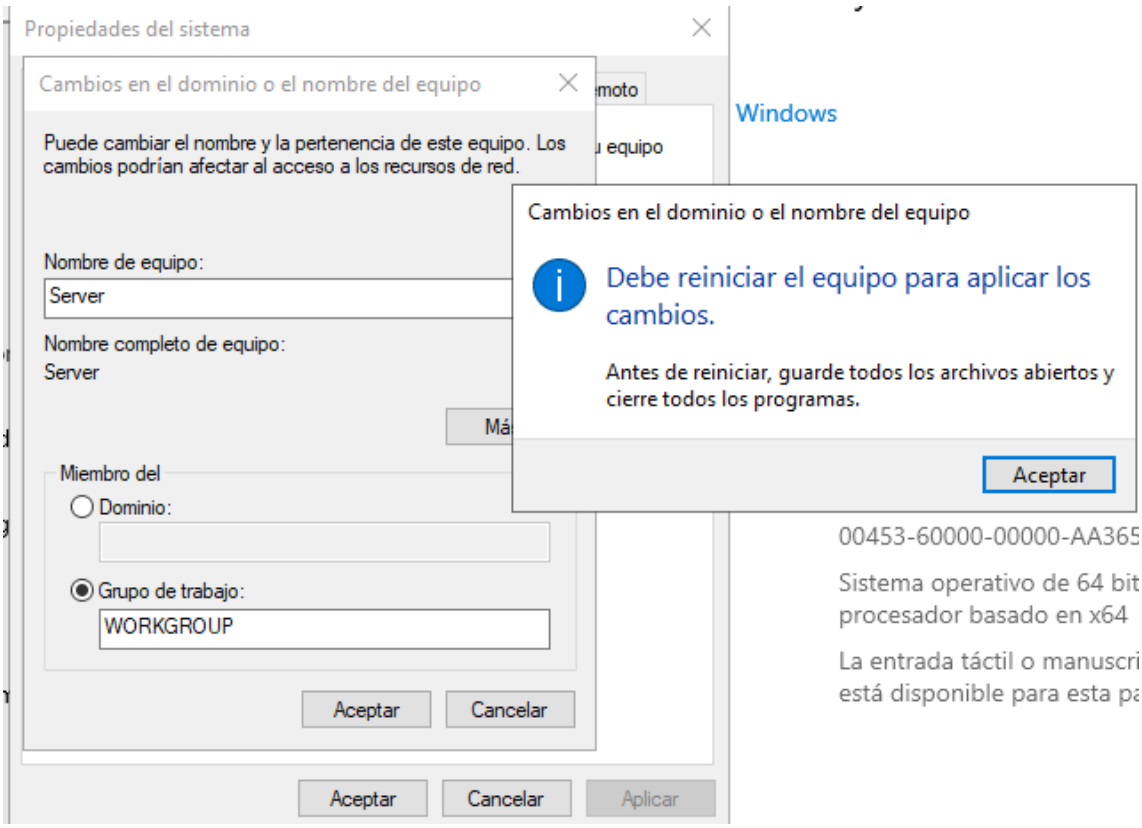


Posteriormente activamos los servicios necesarios en el siguiente orden para que no falle:

- Cliente DNS
- Publicación de recursos de detección de funciones
- Detección de SSDP
- Dispositivo Host de UPnP

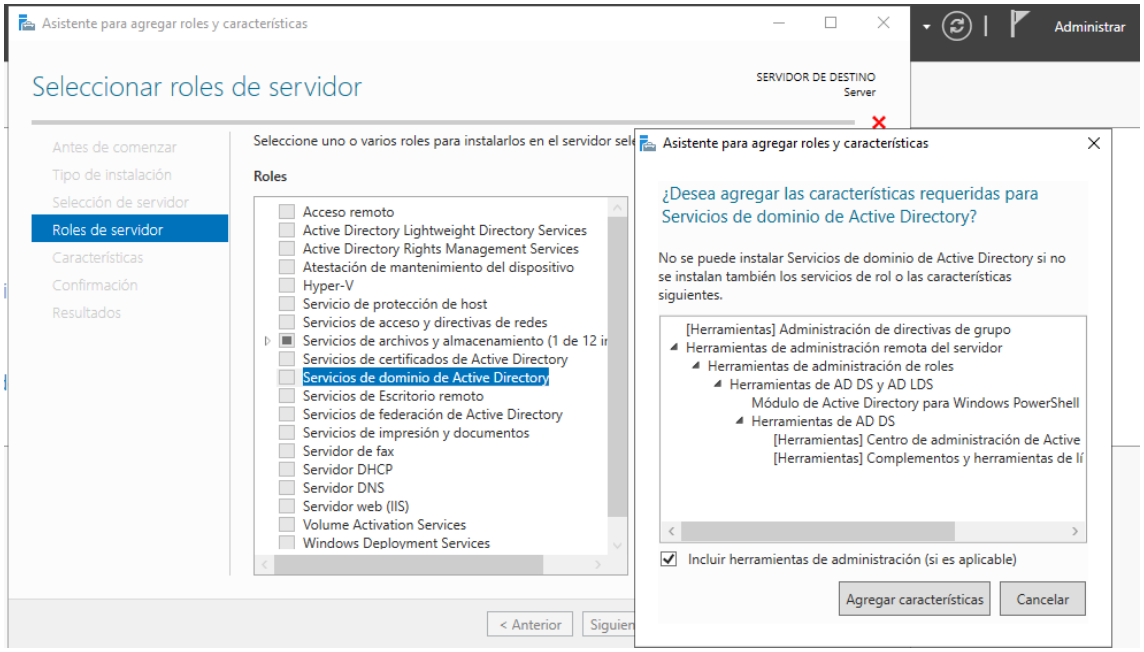
Estos los pondremos en inicio automático e iniciaremos si aún no están activos.

Después debemos cambiar el nombre del equipo:

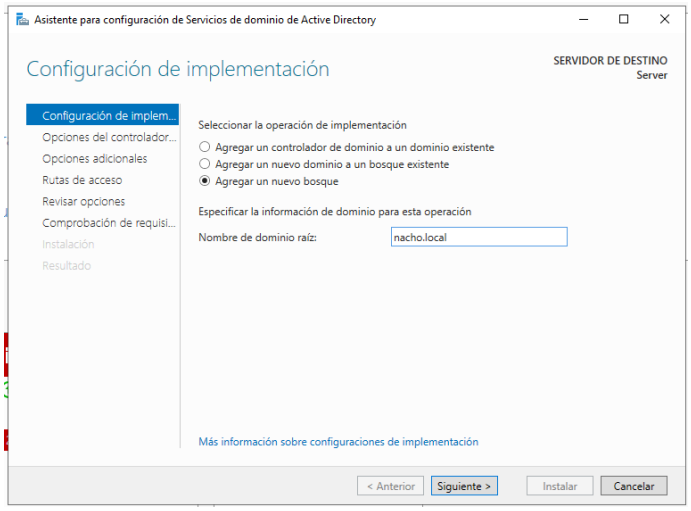


Una vez realizada los pasos anteriores ya podemos instalar el servicio ADDS, para ello vamos al panel de administración del servidor en Administrar-Agregar roles y características.

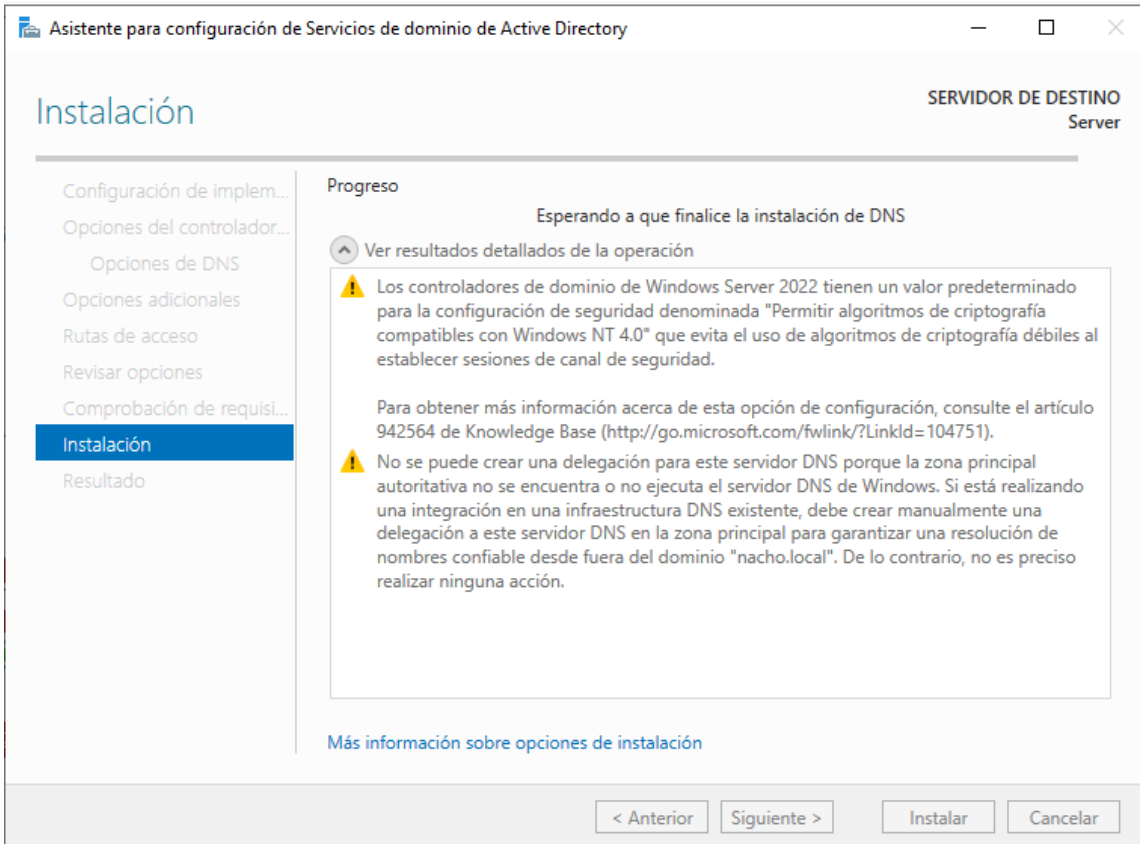
Elegimos una instalación basada en roles y agregamos las características de Active Directory



Cuando la instalación se completa seleccionamos la bandera de notificaciones y promovemos a controlador de dominio. Agregamos a un nuevo bosque en el que ponemos el nombre que deseemos, en nuestro caso “nacho.local”.



Le daremos un par de veces a “Siguiente” de manera que llegaremos a la última pantalla donde comenzará a instalarse todo. Cuando se complete se reiniciará el equipo.

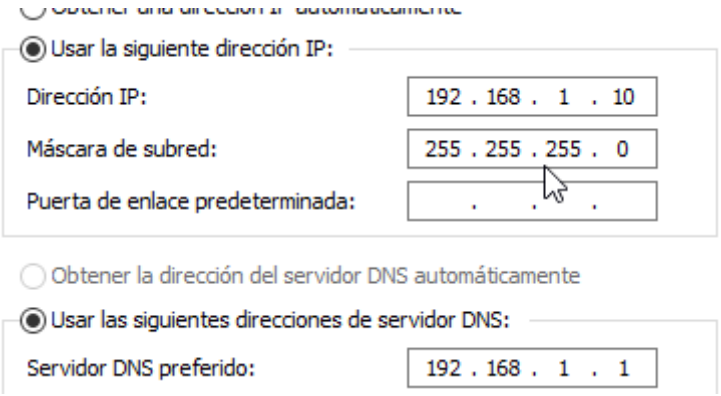


Este es el resumen general de la configuración de la máquina:

3.1.2 Máquina Cliente

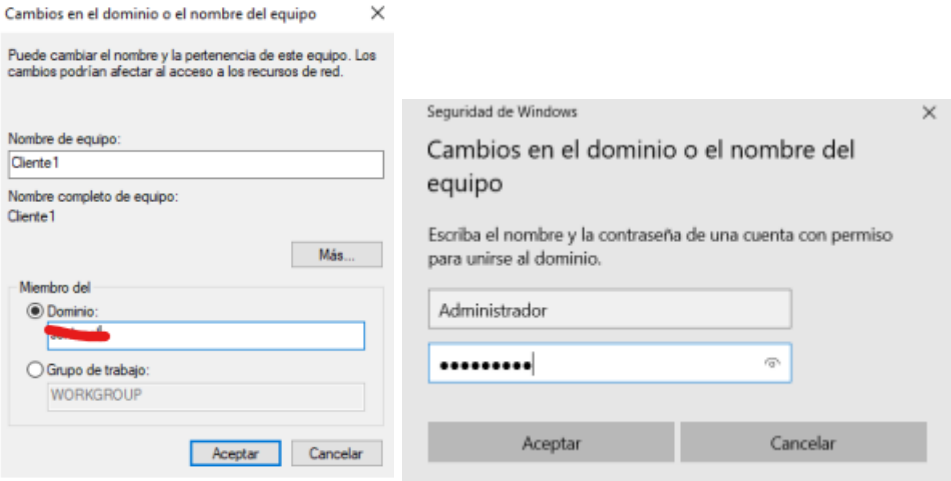
Este paso va a ser más sencillo y no vamos a pararnos a especificar tanto como lo hemos hecho con la máquina servidor.

Iniciamos con una máquina Windows 10 normal. Ponemos una IP fija que coincida dentro de la red que hemos puesto antes y ponemos de DNS la IP del controlador de dominio:



También debemos activar la detección de redes y el uso compartido de archivos e impresoras.

Le cambiamos el nombre al equipo para poder reconocerlo y tras reiniciar añadimos el equipo al dominio nacho.local:



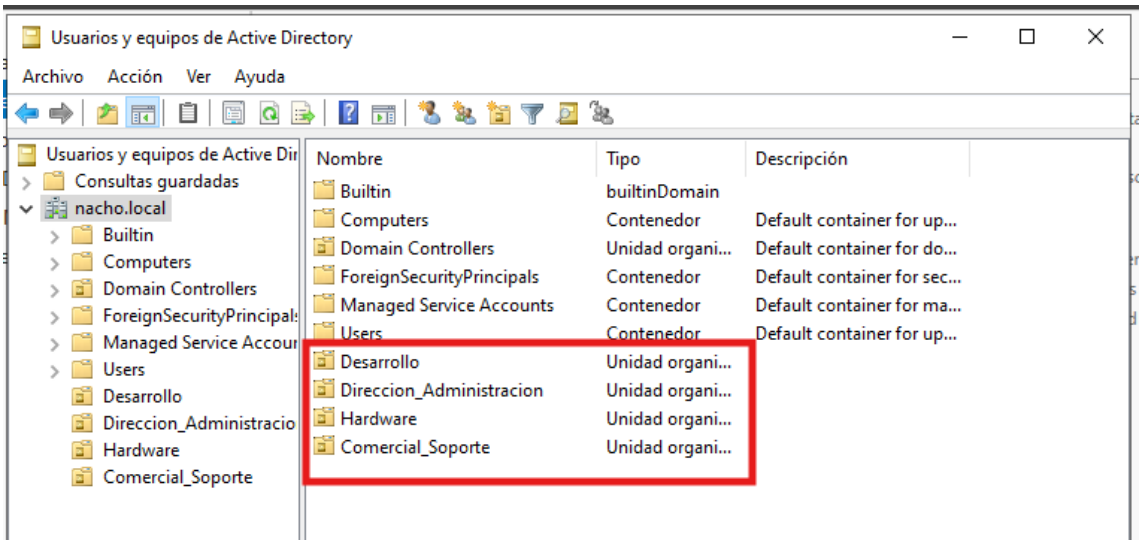
Nos pedirá las credenciales del Administrador. Nos pedirá que se reinicie el equipo y se aplicarán los cambios.

Ya podremos acceder tanto en local, como con un usuario del dominio (la opción que queremos).

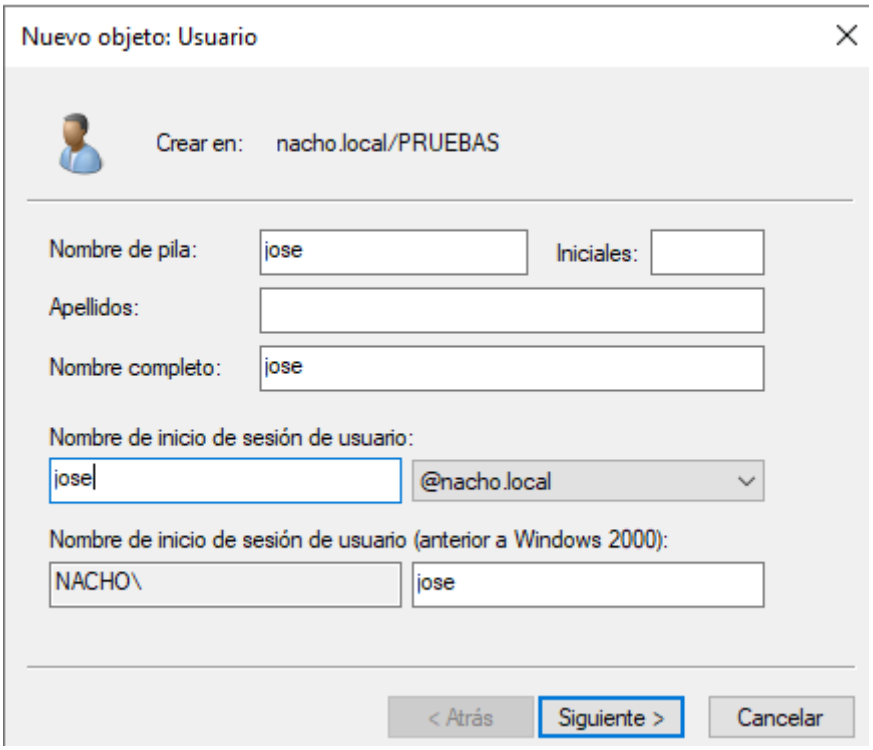
3.1.3 Creación de usuarios

Desde el servidor vamos a crear un par de Unidades Organizativas y usuarios dentro de estas para comprobar el correcto funcionamiento.

Primero creamos las UO que utilizaremos para los usuarios de “una empresa”:

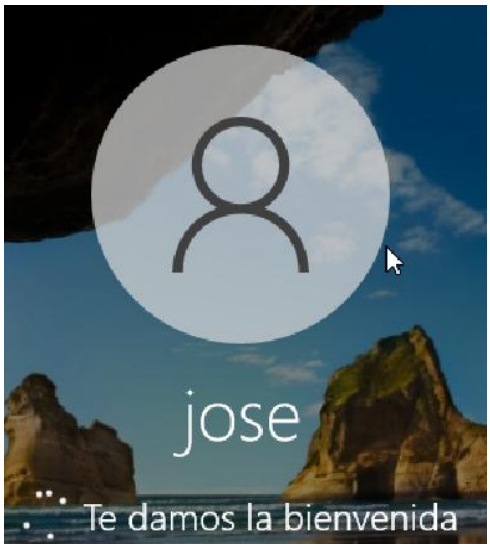


También crearemos una UO de prueba en la que pondremos los usuarios para comprobar que todo funcione:



3.1.4 Comprobaciones

Por último, vamos a entrar con el usuario de prueba creado en el cliente que hemos añadido al dominio:



3.2 Automatización de Gestión de Usuarios

La automatización consistiría en la creación de scripts que gestionen el ciclo de vida completo de los usuarios en Active Directory: onboarding (alta de nuevos empleados), actualización de permisos (cambios de departamento o roles) y offboarding (baja o des habilitación de cuentas).

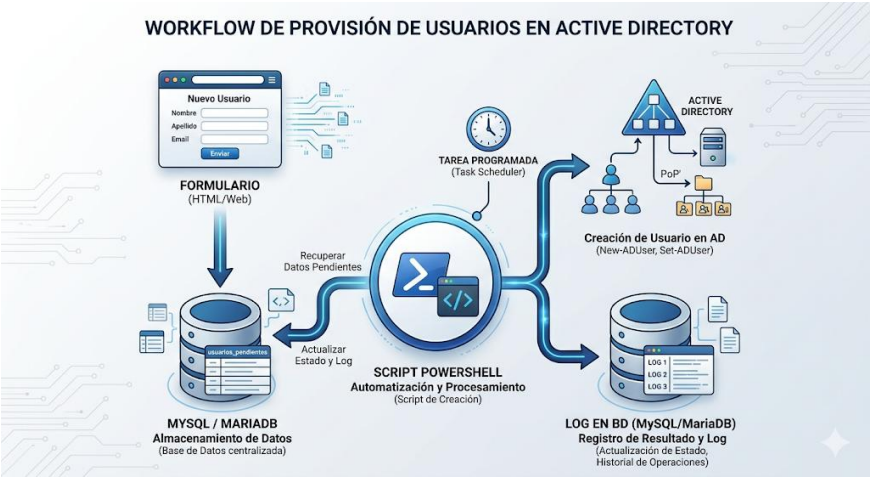
Por ejemplo, al recibir una solicitud de alta (vía formulario simple), el script crearía automáticamente el usuario en la UO correspondiente (p.ej., "Dirección y Administración", "Desarrollo", "Hardware" o "Comercial_Soporte"), asignaría grupos de seguridad predeterminados, configuraría perfiles móviles, aplicaría políticas de grupo (GPO) para restricciones (como bloqueo de USB o herramientas críticas como CMD), generaría una contraseña inicial segura (12-14 caracteres con complejidad) y enviaría notificaciones por email al departamento de RRHH y al nuevo usuario.

Para offboarding, el script deshabilitaría la cuenta, revocaría accesos a recursos compartidos y archivaría el perfil.

Esto se integraría con el servidor principal (server1 con IP estática), aprovechando los servicios existentes de AD, DNS y GPO (si hemos puesto, en nuestro caso hemos personalizado alguna aparte), sin alterar la estructura jerárquica actual.

Este es un esquema básico de cómo debería funcionar:

Formulario (HTML/Web) → MySQL/MariaDB → Script PowerShell → Active Directory → Log en BD

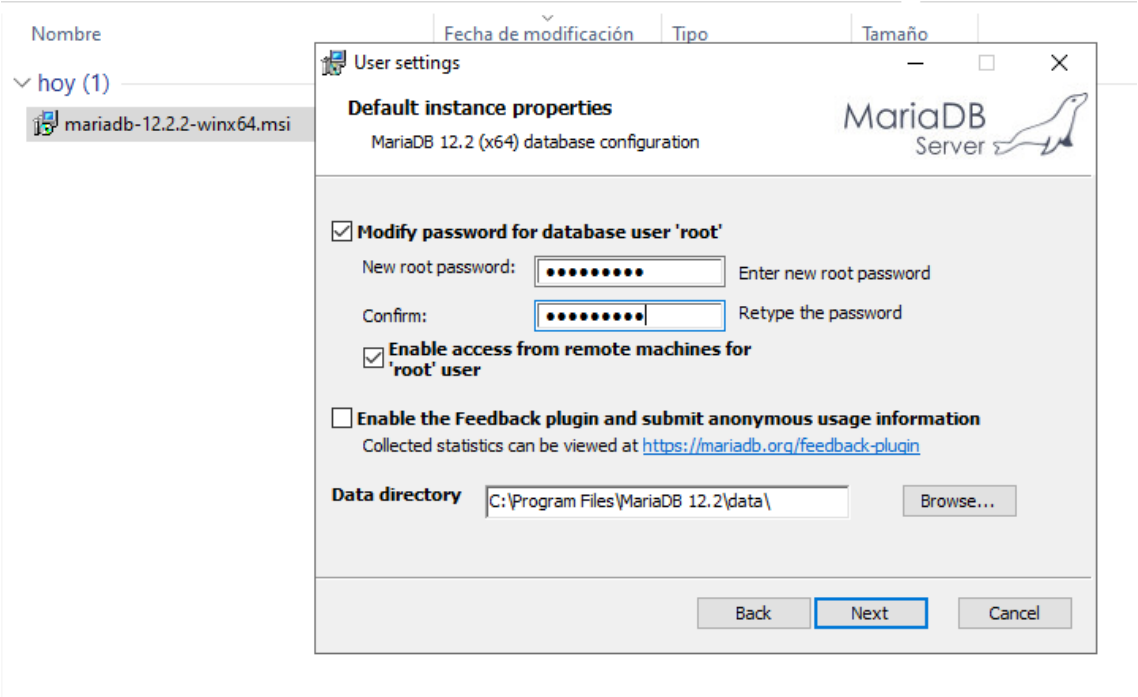


3.2.1 Instalar MySQL/MariaDB en Windows Server

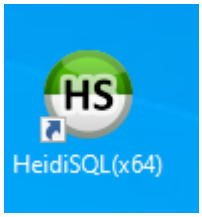
Vamos a descargar la base de datos en formato msi desde el servidor

https://mariadb.org/download/

Y la instalamos y configuramos con una contraseña y parámetros de servicio para que arranque automáticamente.



También se nos instala automáticamente Heidi SQL por lo que accederemos desde ahí para crear la base de datos.



3.2.2 Crear la base de datos y tablas

En una consulta de Heidi SQL ponemos este SQL completo y lo ejecutamos:

```
CREATE DATABASE IF NOT EXISTS gestion_usuarios CHARACTER SET utf8mb4;

USE gestion_usuarios;


CREATE TABLE solicitudes (

    id INT AUTO_INCREMENT PRIMARY KEY,

    tipo ENUM('ALTA', 'MODIFICACION', 'BAJA') NOT NULL,

    nombre VARCHAR(100) NOT NULL,

    apellidos VARCHAR(100) NOT NULL,

    username VARCHAR(50),

    email VARCHAR(150),

    departamento ENUM('Direccion_Administracion', 'Desarrollo', 'Hardware', 'Comercial_Soporte') NOT NULL,

    rol VARCHAR(100),

    fecha_solicitud DATETIME DEFAULT NOW(),

    estado ENUM('PENDIENTE', 'PROCESADO', 'ERROR') DEFAULT 'PENDIENTE',

    fecha_procesado DATETIME NULL,

    notas TEXT NULL

);
```

```
CREATE TABLE auditoria (
    id INT AUTO_INCREMENT PRIMARY KEY,
    solicitud_id INT,
    accion VARCHAR(255),
    resultado ENUM('OK', 'ERROR'),
    detalle TEXT,
    fecha DATETIME DEFAULT NOW(),
    FOREIGN KEY (solicitud_id) REFERENCES solicitudes(id)
);

CREATE TABLE usuarios_activos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    nombre_completo VARCHAR(200),
    email VARCHAR(150),
    departamento VARCHAR(100),
    fecha_alta DATETIME DEFAULT NOW(),
    activo BOOLEAN DEFAULT TRUE
);
```

Ahora ya tenemos las 3 tablas que necesitamos creadas:

Unnamed | Base de datos: mysql | Consulta*

Nombre	Filas	Tamaño	Creado	Actualizado	Motor	Comentari
auditoria	0	32,0 KiB	2026-03-12 06:49:56		InnoDB	
solicitudes	0	16,0 KiB	2026-03-12 06:49:56		InnoDB	
usuarios_activos	0	32,0 KiB	2026-03-12 06:49:56		InnoDB	

3.2.3 Instalar el conector MySQL para PowerShell y crear Scripts

Creamos una carpeta de trabajo para poner los scripts e instalamos un módulo de compatibilidad entre mariadb y powershell en nuestro caso SimplySql. Lo vamos a instalar con este comando desde el PowerShell “Install-Module -Name SimplySql -Force -AllowClobber”. Y comprobamos que se ha instalado correctamente:

```
PS C:\Users\Administrador> Get-Module -ListAvailable SimplySql
>>


    Directorio: C:\Program Files\WindowsPowerShell\Modules


ModuleType Version    Name                                ExportedCommands
---------- -------    ----                                ----------------
Binary     2.1.0.96   SimplySql                           {Clear-SqlMessage, Close-SqlConnection, Complete-SqlT...
```

Dentro de la carpeta de trabajo (C:\Scripts\GestionUsuarios) creamos 4 scripts, con el siguiente flujo de trabajo en mente de momento:

MariaDB (solicitudes)

↓

config.ps1 + funciones.ps1 (base común)

↓

procesar_altas.ps1 → crea en AD → escribe en auditoria

procesar_bajas.ps1 → deshabilita en AD → escribe en auditoria

Hemos probado el script de alta y funciona perfectamente, los usuarios se crean en las UO correspondientes :

Creamos la solicitud en la base de datos desde una consulta de Heidi SQL:

INSERT INTO solicitudes (tipo, nombre, apellidos, email, departamento, rol)

VALUES ('ALTA', 'Prueba', 'Jefe', 'pruebajefe@nacho.local', 'Direccion_Administracion', 'Director');

Observamos que la solicitud se ha tramitado y ejecutamos el script.

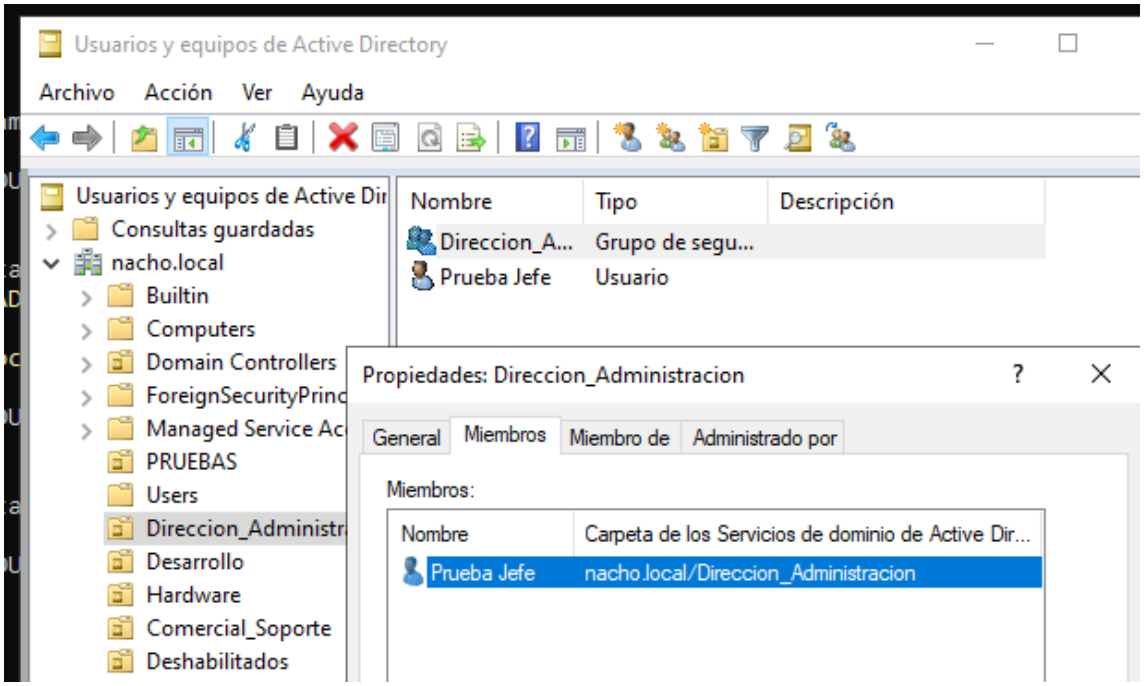
```
1  SELECT id, nombre, estado, tipo FROM solicitudes;
2
```

solicitudes (1r × 4c)

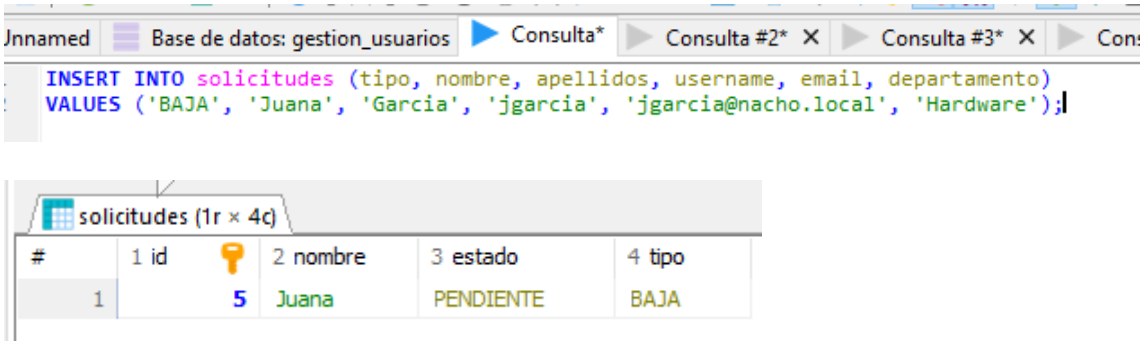
#	1 id	2 nombre	3 estado	4 tipo
1	3	Prueba	PENDIENTE	ALTA

```
PS C:\Scripts\GestionUsuarios> .\procesar_altas.ps1
Procesando ALTA para: Prueba Jefe
DEBUG - username: 'pjefe' | ou: 'OU=Direccion_Administracion,DC=nacho,DC=local'
1
1
1
1
OK - Usuario pjefe creado correctamente
PS C:\Scripts\GestionUsuarios>
```

Vemos que el script se ejecuta correctamente, vamos a comprobar si se ha creado el usuario desde la interfaz gráfica:



Y ahora probamos con el script de bajas, insertando la solicitud de baja de un usuario ya creado:

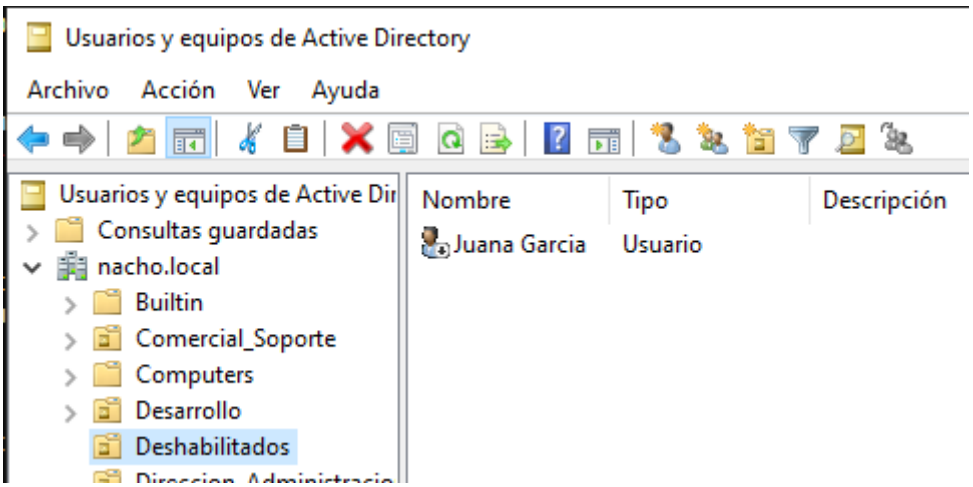


Ejecutamos el script:

```
PS C:\Scripts\GestionUsuarios> .\procesar_bajas.ps1
Procesando BAJA para: Juana Garcia (usuario: jgarcia)
1
1
1
OK - Baja de jgarcia procesada correctamente
```

Comprobamos que se haya deshabilitado correctamente:

Desde interfaz gráfica



Desde comandos

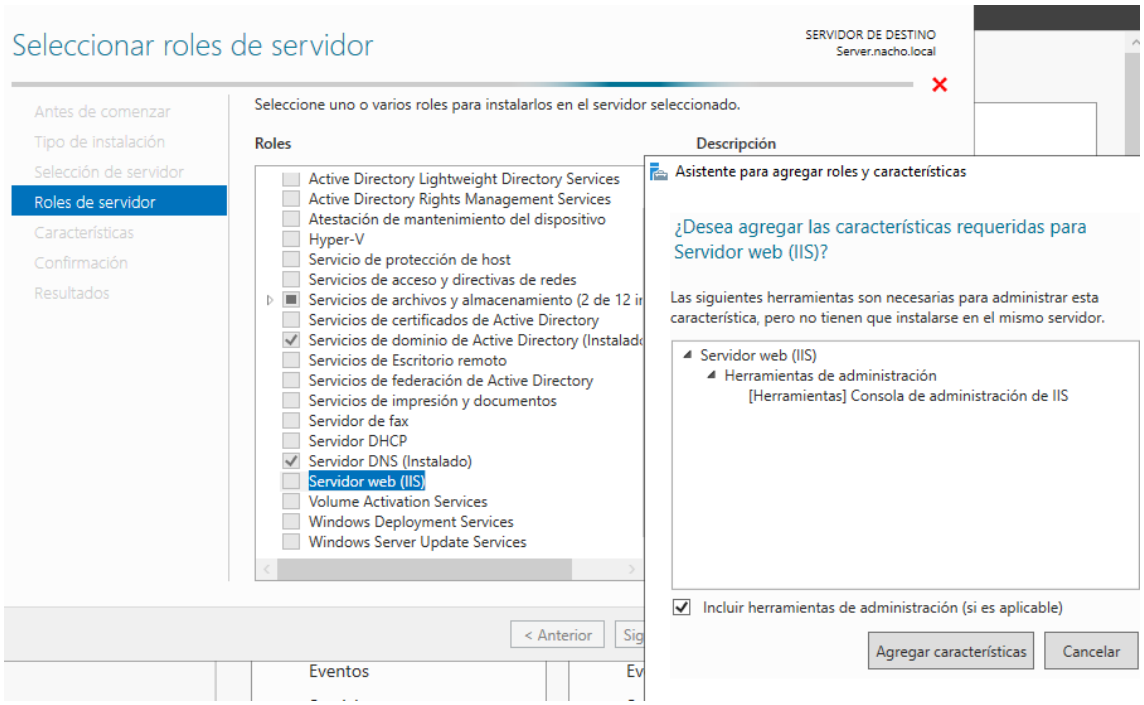
```
PS C:\Scripts\GestionUsuarios> Get-ADUser -LDAPFilter "(sAMAccountName=jgarcia)" | Select-Object Name, Enabled, D
tinguishedName

Name         Enabled DistinguishedName
----         ------- -----------------
Juana Garcia   False CN=Juana Garcia,OU=Deshabilitados,DC=nacho,DC=local
```

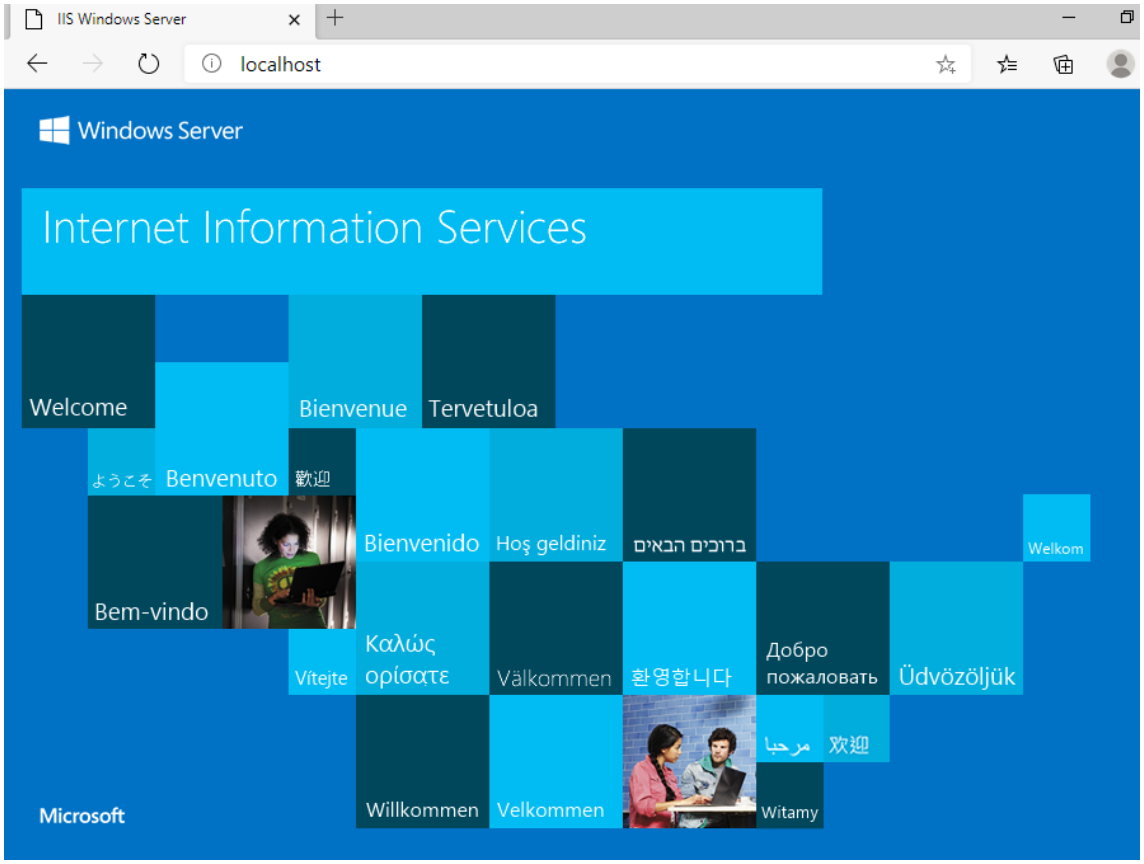
3.2.4 Formulario Web y conectividad con la base de datos

Hemos creado un Formulario para las personas de Recursos Humanos que recoja la solicitud de Baja o Alta y le pase los datos a otro archivo PHP que será el que se lo transmita a la base de datos.

Debemos instalar php y un servidor web. Vamos a aprovechar que Windows server tiene IIS y vamos a activarlo como servidor web:



Verificamos que haya funcionado desde el navegador dándonos la página de bienvenida



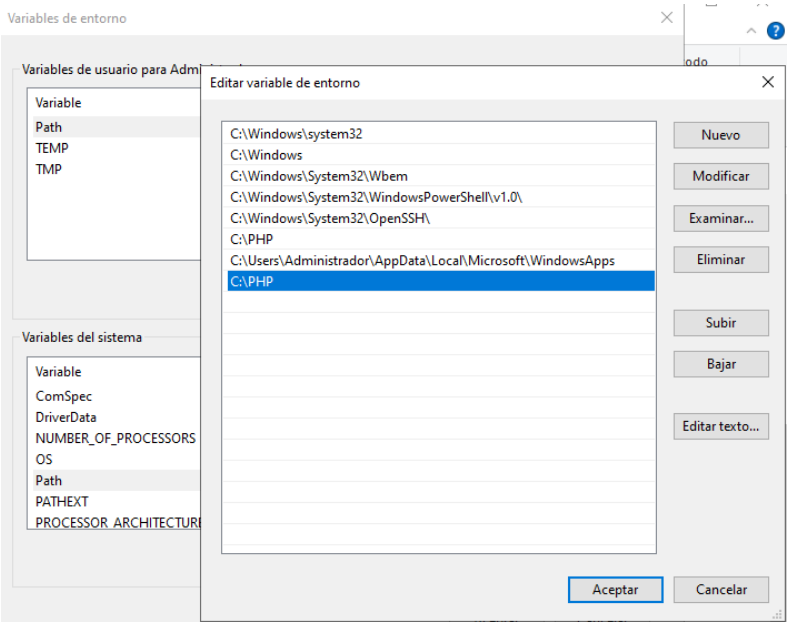
Ahora instalamos php descargándonos el archivo y descomprimiéndolo en C:\PHP (si no existe, lo creamos). Buscamos un archivo llamado “php.ini-development” lo copiamos y le cambiamos el nombre a php.ini des comentando tres líneas dentro de este:

extension=pdo_mysql

extension=mysqli

extension=mbstring

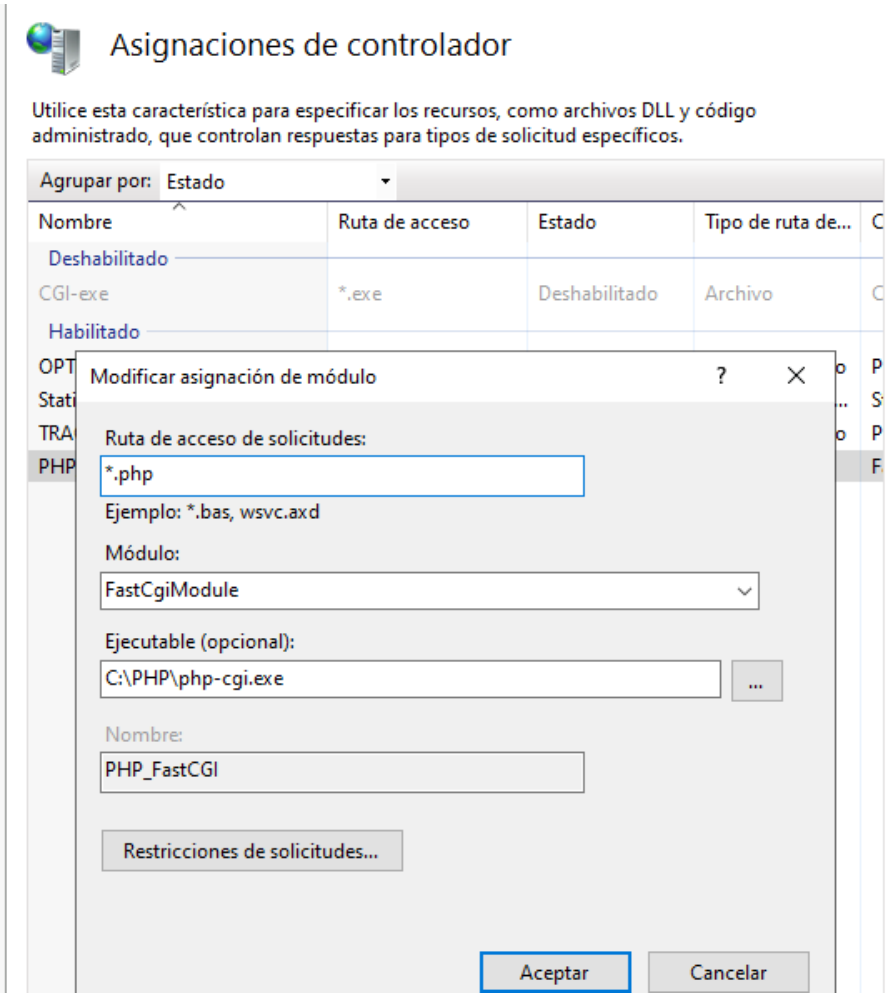
También hemos añadido la ruta a esta carpeta en las variables de entorno de Windows, en sistema, path.



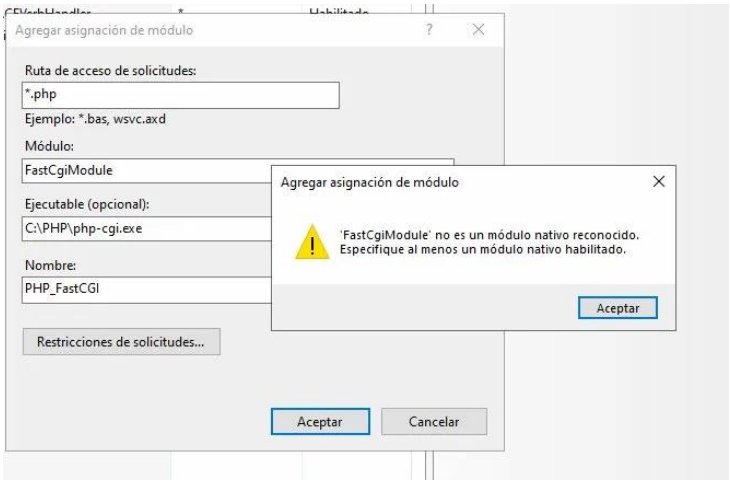
Comprobamos la versión de php para ver que todo ha salido bien.

```
PS C:\Users\Administrador> php -v
PHP 8.5.1 (cli) (built: Dec 17 2025 10:54:30) (NTS Visual C++ 2022 x64)
Copyright (c) The PHP Group
Zend Engine v4.5.1, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.1, Copyright (c), by Zend Technologies
PS C:\Users\Administrador>
```

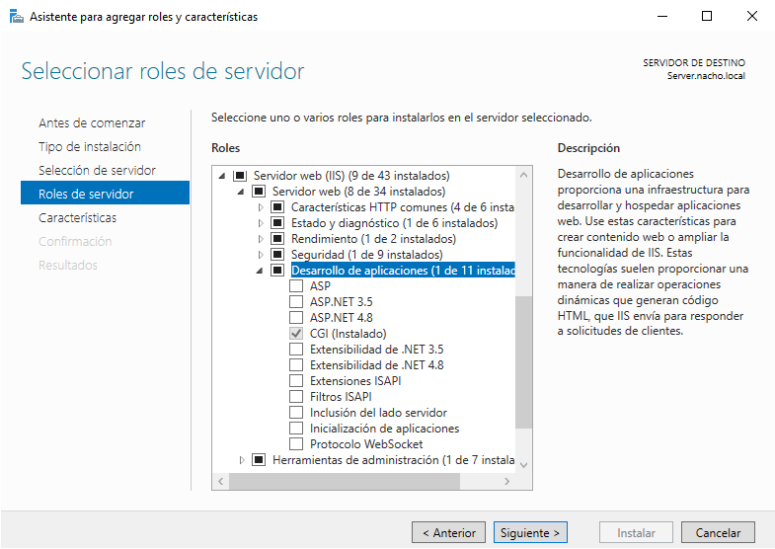
Con php ya instalado, debemos de hacer que IIS lo reconozca. Entramos desde la aplicación de ISS y en asignaciones del controlador añadimos un módulo:



Puede que no nos deje si no tenemos FastCgiModule instalado en ISS y tengamos que activarlo. Nos dará este error:

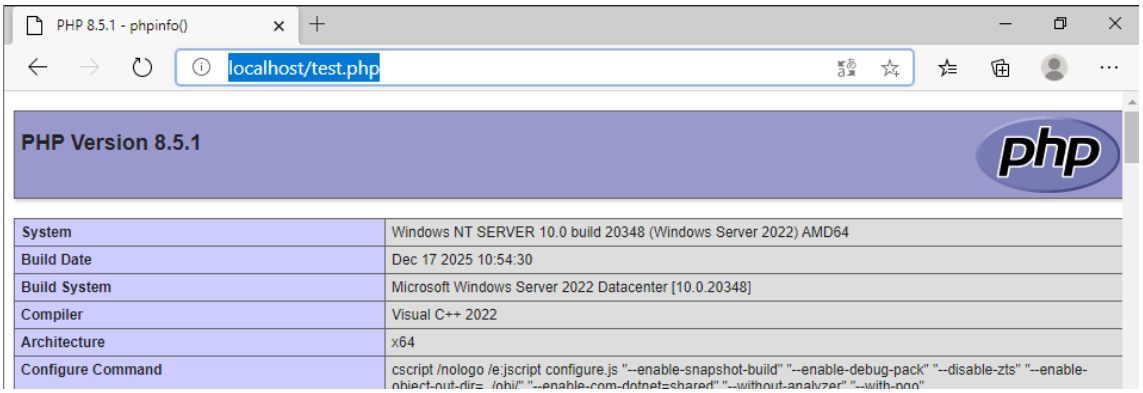


Al instalar CGI también lo hace FastCGI:

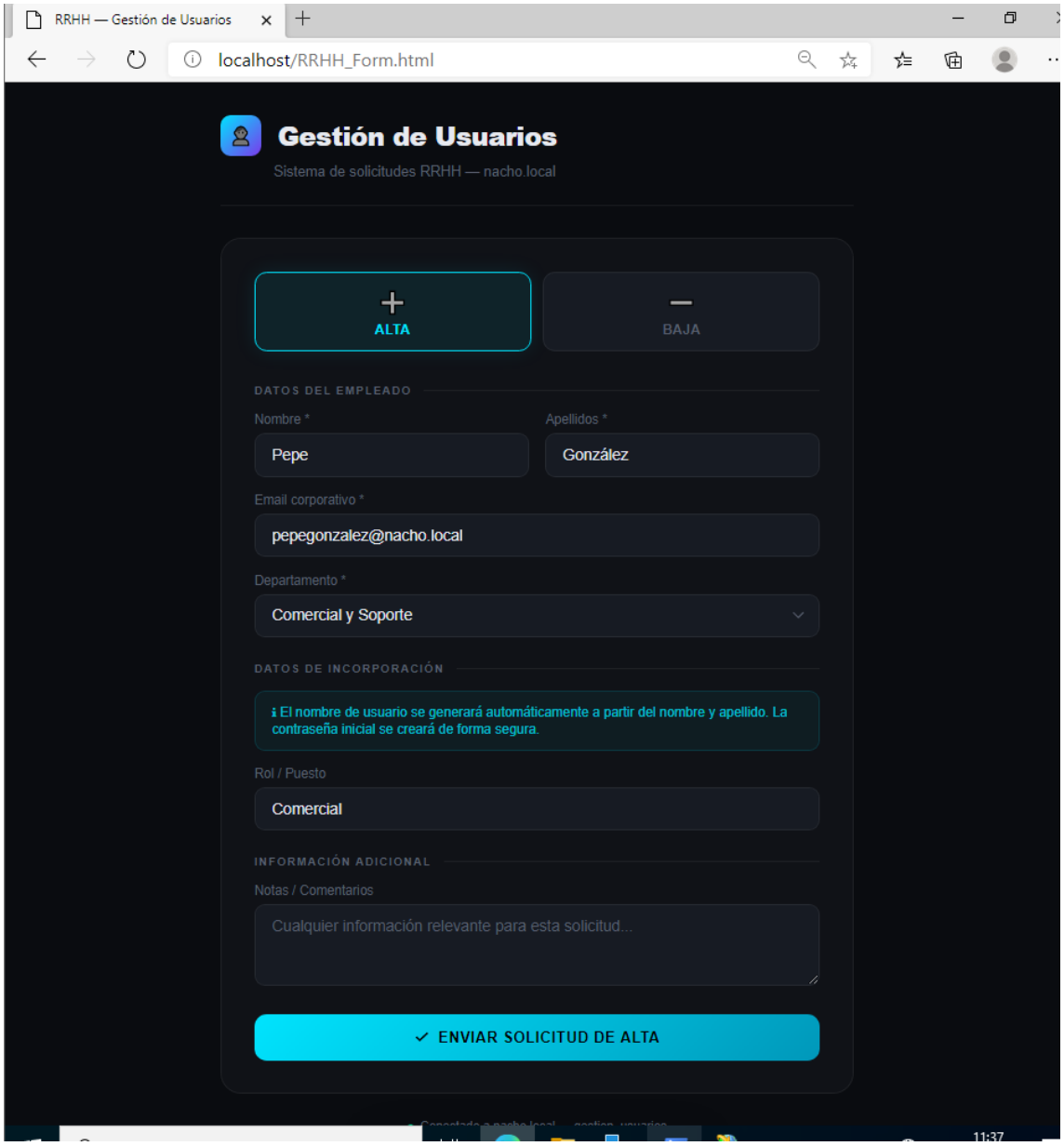


Creamos un archivo de prueba desde PowerShell para visualizar y probamos que se vea desde el navegador abriendo “http://localhost/test.php”

```
PS C:\Users\Administrador> "<?php phpinfo(); ?>" | Out-File -FilePath "C:\inetpub\wwwroot\test.php" -Encoding UTF8
```



Ya con PHP y ISS funcionando, copiamos los archivos de formulario en .html y de receptor en .php a "C:\inetpub\wwwroot\. Y comprobamos el funcionamiento en el navegador con el siguiente enlace http://localhost/RRHH_Form.html:



La solicitud se inserta en la base de datos gracias al formulario web. Ahora vamos a ejecutar el script para que se procese correctamente:

```
PS C:\Scripts\GestionUsuarios> .\procesar_altas.ps1
Procesando ALTA para: Pepe González
DEBUG - username: 'pgonzález' | ou: 'OU=Comercial_Soporte,DC=nacho,DC=local'
1
1
1
1
OK - Usuario pgonzález creado correctamente
PS C:\Scripts\GestionUsuarios>
```

Pero para que sea completamente automático debemos hacer una tarea programada haciendo que este Script se ejecute sin interacción manual.

3.2.5 Programador de Tareas

Para ello creamos las tareas de alta y baja desde la terminal:

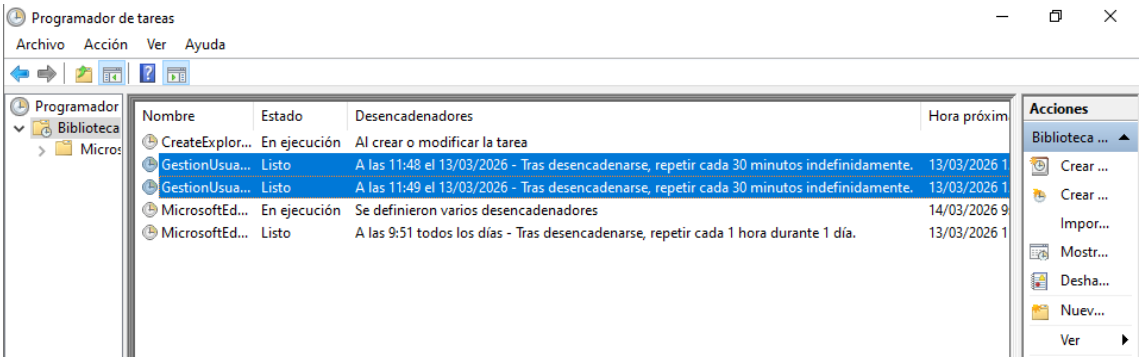
```
PS C:\Scripts\GestionUsuarios> $accion = New-ScheduledTaskAction -Execute "powershell.exe" `
>>     -Argument "-ExecutionPolicy Bypass -NonInteractive -File C:\Scripts\GestionUsuarios\procesar_altas.ps1"
>>
>> $trigger = New-ScheduledTaskTrigger -RepetitionInterval (New-TimeSpan -Minutes 30) -Once -At (Get-Date)
>>
>> $settings = New-ScheduledTaskSettingsSet -ExecutionTimeLimit (New-TimeSpan -Minutes 5) -RunOnlyIfNetworkAvailabl
e
>>
>> Register-ScheduledTask -TaskName "GestionUsuarios_Altas" `
>>     -Action $accion `
>>     -Trigger $trigger `
>>     -Settings $settings `
>>     -RunLevel Highest `
>>     -User "SYSTEM" `
>>     -Description "Procesa solicitudes de alta de usuarios en Active Directory"

TaskPath                                       TaskName                       State
--------                                       --------                       -----
\                                              GestionUsuarios_Altas          Ready
```

```
PS C:\Scripts\GestionUsuarios> $accion2 = New-ScheduledTaskAction -Execute "powershell.exe" `
>>     -Argument "-ExecutionPolicy Bypass -NonInteractive -File C:\Scripts\GestionUsuarios\procesar_bajas.ps1"
>>
>> $trigger2 = New-ScheduledTaskTrigger -RepetitionInterval (New-TimeSpan -Minutes 30) -Once -At (Get-Date)
>>
>> $settings2 = New-ScheduledTaskSettingsSet -ExecutionTimeLimit (New-TimeSpan -Minutes 5) -RunOnlyIfNetworkAvailab
le
>>
>> Register-ScheduledTask -TaskName "GestionUsuarios_Bajas" `
>>     -Action $accion2 `
>>     -Trigger $trigger2 `
>>     -Settings $settings2 `
>>     -RunLevel Highest `
>>     -User "SYSTEM" `
>>     -Description "Procesa solicitudes de baja de usuarios en Active Directory"

TaskPath                                       TaskName                       State
--------                                       --------                       -----
\                                              GestionUsuarios_Bajas          Ready
```

Si vemos desde la interfaz gráfica, estas tareas provocan que se ejecute el script cada 30 minutos.



Ya tenemos todo el flujo completado. Por último, vamos a añadir otro script que borre las solicitudes (limpiar_procesadas.ps1) ya procesadas con una tarea que se ejecute cada 24 horas y borre las solicitudes en las que ya haya pasado más de 24 horas.

```
PS C:\Users\Administrador> $accionLimpieza = New-ScheduledTaskAction -Execute "powershell.exe" `
>>     -Argument "-ExecutionPolicy Bypass -NonInteractive -File C:\Scripts\GestionUsuarios\limpiar_procesadas.ps1"
>>
>> $triggerLimpieza = New-ScheduledTaskTrigger -Daily -At "03:00"
>>
>> $settingsLimpieza = New-ScheduledTaskSettingsSet -ExecutionTimeLimit (New-TimeSpan -Minutes 5)
>>
>> Register-ScheduledTask -TaskName "GestionUsuarios_Limpieza" `
>>     -Action $accionLimpieza `
>>     -Trigger $triggerLimpieza `
>>     -Settings $settingsLimpieza `
>>     -RunLevel Highest `
>>     -User "SYSTEM" `
>>     -Description "Borra solicitudes procesadas con más de 24 horas"
```

GestionUsua...	Listo	A las 3:00 todos los días	14/03/2026 3

Este es en resumen el flujo completo de la automatización y mejora realizada:

1. Insertas solicitud en MySQL (a través del formulario web)

   ↓

2. Task Scheduler ejecuta los scripts cada X tiempo

   ↓

3. 3. PowerShell lee solicitudes PENDIENTES de la BD

   ↓

4. Ejecuta acciones en Active Directory (New-ADUser, Disable-ADAccount...)

   ↓

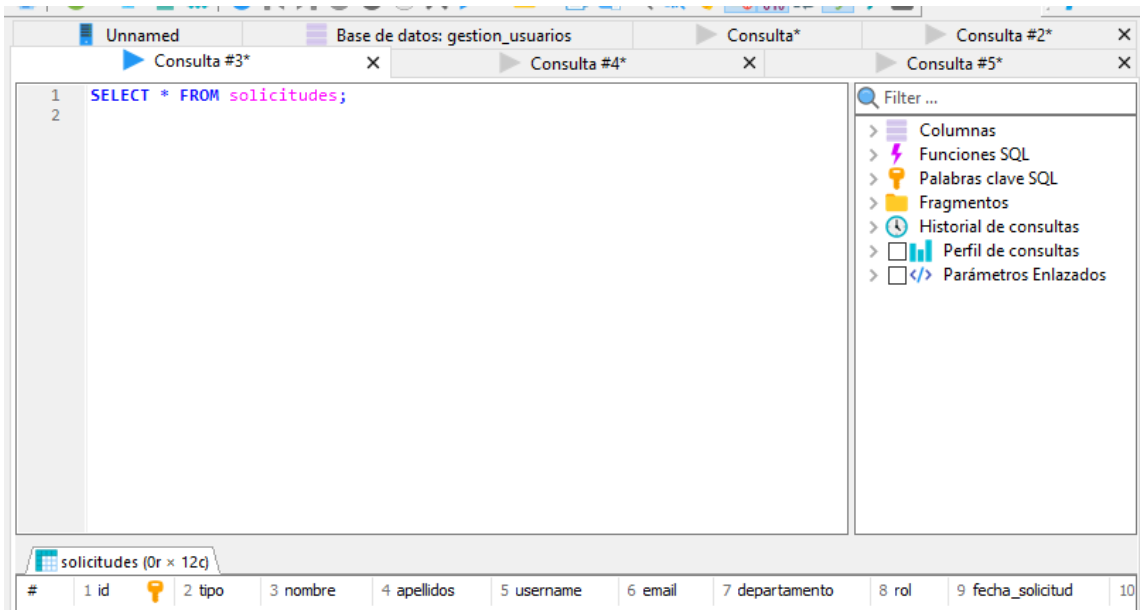
5. Escribe el resultado en tabla "auditoria"

   ↓

6. Marca la solicitud como PROCESADO o ERROR

Pero comprobamos su ejecución de manera inmediata:

```
PS C:\Scripts\GestionUsuarios> .\limpiar_procesadas.ps1
2
Limpieza completada - Solicitudes borradas: 2
PS C:\Scripts\GestionUsuarios>
```



Podríamos añadir una opción de modificación de los usuarios, por si hay cambios de departamento, pero se ha decidido que no en caso de error humano.

3.2.6 Resolución de Errores

A lo largo de la prueba y error de scripts, desde Heidi SQL hemos estado insertando, actualizando y borrando solicitudes de esta manera:

Insertamos solicitudes:

```
INSERT INTO solicitudes (tipo, nombre, apellidos, email, departamento, rol)
VALUES ('ALTA', 'Laura', 'García López', 'lgarcia@techsolutions.local', 'Desarrollo', 'Desarrollado
```

Revisamos solicitudes:

```
SELECT id, nombre, estado, tipo FROM solicitudes;
```

```
SELECT * FROM solicitudes;
```

Con su resultado:

solicitudes (1r × 4c)

#	1 id	2 nombre	3 estado	4 tipo
1	4	Laura	PENDIENTE	ALTA

solicitudes (1r × 12c)

#	1 id	2 tipo	3 nombre	4 apellidos	5 username	6 email	7 departamento	8 rol
1	4	ALTA	Laura	García López	(NULL)	lgarcia@techsolutions.local	Desarrollo	Desarrolladora

Borramos solicitudes ya procesadas:

```
DELETE FROM auditoria WHERE solicitud_id IN (SELECT id FROM solicitudes WHERE estado='PROCESADO');
DELETE FROM solicitudes WHERE estado='PROCESADO';
```

Dentro de los scripts tanto en el de alta como el de baja hemos tenido que modificar una línea por el mismo error. Cuando se realizaba la solicitud y ejecutábamos nuestro script se realizaba un error en el tipo (Alta/Baja) y estado de solicitud (Pendiente/Procesada/Error). Debe estar antes el tipo que el estado para que no falle, sino nos dirá que se ha realizado, pero no ha funcionado.

Esta es la línea errónea:

$solicitudes = Invoke-SqlQuery "SELECT id, tipo, nombre, apellidos, username, email, departamento FROM solicitudes **WHERE estado='PENDIENTE' AND tipo='BAJA'"**

Esta es la línea corregida:

$solicitudes = Invoke-SqlQuery "SELECT id, tipo, nombre, apellidos, username, email, departamento FROM solicitudes **WHERE tipo='BAJA' AND estado='PENDIENTE'"**

4. Conclusión

En este proyecto se ha conseguido implementar un entorno funcional de dominio Windows junto con un sistema automatizado de gestión de usuarios, integrando diferentes tecnologías de forma coherente. Se ha pasado de una configuración manual básica de Active Directory a un modelo más avanzado donde las altas y bajas de usuarios pueden gestionarse de manera automatizada mediante scripts y tareas programadas.

Uno de los puntos más relevantes ha sido la integración entre la base de datos, los scripts de PowerShell y el entorno de Active Directory, logrando un flujo completo desde la solicitud hasta la ejecución y registro de resultados. También ha sido clave la fase de pruebas y resolución de errores, que ha permitido ajustar detalles importantes del sistema.

Como posibles mejoras, se podrían añadir funcionalidades como la modificación de usuarios, una interfaz web más avanzada o mayor control de validaciones y seguridad. Aun así, el proyecto cumple su objetivo principal: demostrar cómo automatizar procesos administrativos en un entorno de red de forma práctica y escalable.

5. Bibliografía

https://windows.php.net/download/#php-8.5-nts-vs17-x64

https://mariadb.org/download/?t=mariadb&p=mariadb&r=12.2.2&os=windows&cpu=x86_64&pkg=msi&mirror=raiolanetworks

https://mariadb.com/docs/platform/post-download/mariadb-server-12.2.2